wsdm-cup-multilingual-chatbot-arena

4th Place Solution

Rank	4
Team	HKUST-GZ DSA KiMi Lab
Author	HengweiDAI
Topic ID	568522
Version	Original

Source: https://www.kaggle.com/competitions/wsdm-cup-multilingual-chatbot-arena/discussion/568522
Retrieved with Kaggle CLI on 2026-10-21.

🙇🙇

Thanks to the sincere sharing from community friends, and special thanks to @sayoulala for sharing the pipeline in the last LMSYS competition. My respects go out to every passionate contestant and congratulations to all the winners who have earned honors! I would also like to extend a special thank you to the KIMI Lab from the DSA Thrust at Hong Kong University of Science and Technology (Guangzhou) for providing **40*A100 80GB GPUs support!** @HKUST-gz

My solution might be relatively simple, especially compared to other outstanding contestants. However, this is the first time I've won a solo gold medal. Compared to the joy of receiving honors, the frustrations and challenges during the competition accounted for 90%. I hope everyone will not hesitate to give me guidance, thank you!

Table of Contents

- TL;DR
- Solution Details
  - CoT as Initial Prompt Control for Outputs A and B
  - AutoModelForSequenceClassification ——> AutoModelForCasualLM
  - Post-Pretrain Improvements
  - Why Stage-wise Design of Training Loss
  - Obtaining Specific Token Logits with vllm, Using allowed_token_ids=[a_tok_id,b_tok_id] Alongside logprobs=N
  - Using GPTQ 8-bit as the Final Quantization Solution
- Some Failed Attempts
- Ideas That Were Not Realized
- Summary

TL;DR

1. AutoModelForCasualLM

(1) Use AutoModelForCasualLM + vllm inference to replace AutoModelForSequenceClassification + transformers inference.

(2) Compare the logits of Token A and Token B to determine the output.

(3) Use Chain-of-Thought (CoT) as the initial prompt.

2. Differences Between Post-Pretrain and Finetune

(1) Dataset Differences: ultrafeedback + C4AI-Community/multilingual-reward-bench (for Post-Pretrain), lmsys (excluding data labeled as tie) + wsdm (for Finetune).

(2) Loss Calculation Differences: During pretraining, use cross-entropy loss of A or B relative to the entire vocabulary; during finetuning, use cross-entropy loss between A and B.

(3) Data Augmentation Differences: In pretraining, use responseA+B and responseB+A within the same batch; in finetuning, only use responseA+B.

(4) Input Length Differences: Use 1024 tokens for pretraining, and 2048 tokens for finetuning.

3. Distillation Optimization

(1) Use the same procedure to separately train Athene-v2-chat + nvidia/Llama-3.1-Nemotron-70B-Instruct-HF + Qwen2.5-72B-Instruct on fine-tuning datasets to generate soft labels.

(2) During finetuning, train for more than one epoch, specifically two epochs. Cross-validation results will show significant improvements (~0.002 average improvement) towards the end of the second epoch.

4. Inference Optimization

(1) Obtain specific token logits using vllm with allowed_token_ids=[a_tok_id,b_tok_id] alongside logprobs=N.

(2) Replace awq with gptq.

Solution Details

CoT as Initial Prompt Control for Outputs A and B

```
():
   messages = [
       {
           : ,
           :
       },
       {
           : ,
           :
       }
   ]
   text = tokenizer.apply_chat_template(messages, tokenize=, add_generation_prompt=)
    text+
```

AutoModelForSequenceClassification ——> AutoModelForCasualLM

- Modify the tokenizer

```
():
   tokenizer = AutoTokenizer.from_pretrained(
       path,
       add_eos_token=,)
   tokenizer.padding_side =
    tokenizer
```

- During finetuning, only use the logits of Token A and Token B to compute the binary classification loss
- Label Mapping：A ——> 0, B ——> 1

```
(nn.Module):
    ():
       ().__init__()
       .model = base_model
       .token_ids = []
        letter  [,]:
           token_id = tokenizer(letter, add_special_tokens=)[][-]
           .tok_locations.append(token_id)
    ():
       outputs = .model(input_ids=input_ids, attention_mask=attention_mask, output_hidden_states=)
       scores = []
        token_id  .tok_locations:
           score = outputs.logits[:, -, token_id]
           scores.append(score)
       logits = torch.stack(scores, )
        logits.contiguous()
    ():
       logits = .encode(input_ids, attention_mask)
       ce_loss = (.loss_fn(logits, labels)).mean()
```

Post-Pretrain Improvements:：

• Compute the cross-entropy loss using the logits of Token A and Token B relative to the entire vocabulary

```
(nn.Module):
    ():
       ().__init__()
       .model = base_model
       .token_ids = []
        letter  [,]:
           token_id = tokenizer(letter, add_special_tokens=)[][-]
           .tok_locations.append(token_id)
    ():
       outputs = .model(input_ids=input_ids, attention_mask=attention_mask, output_hidden_states=)
       logits = outputs.logits[:, -, :]
       ce_loss = (.loss_fn(logits, labels)).mean()
```

• label Mapping：0 ——> A_token_id, 1——> B_token_id

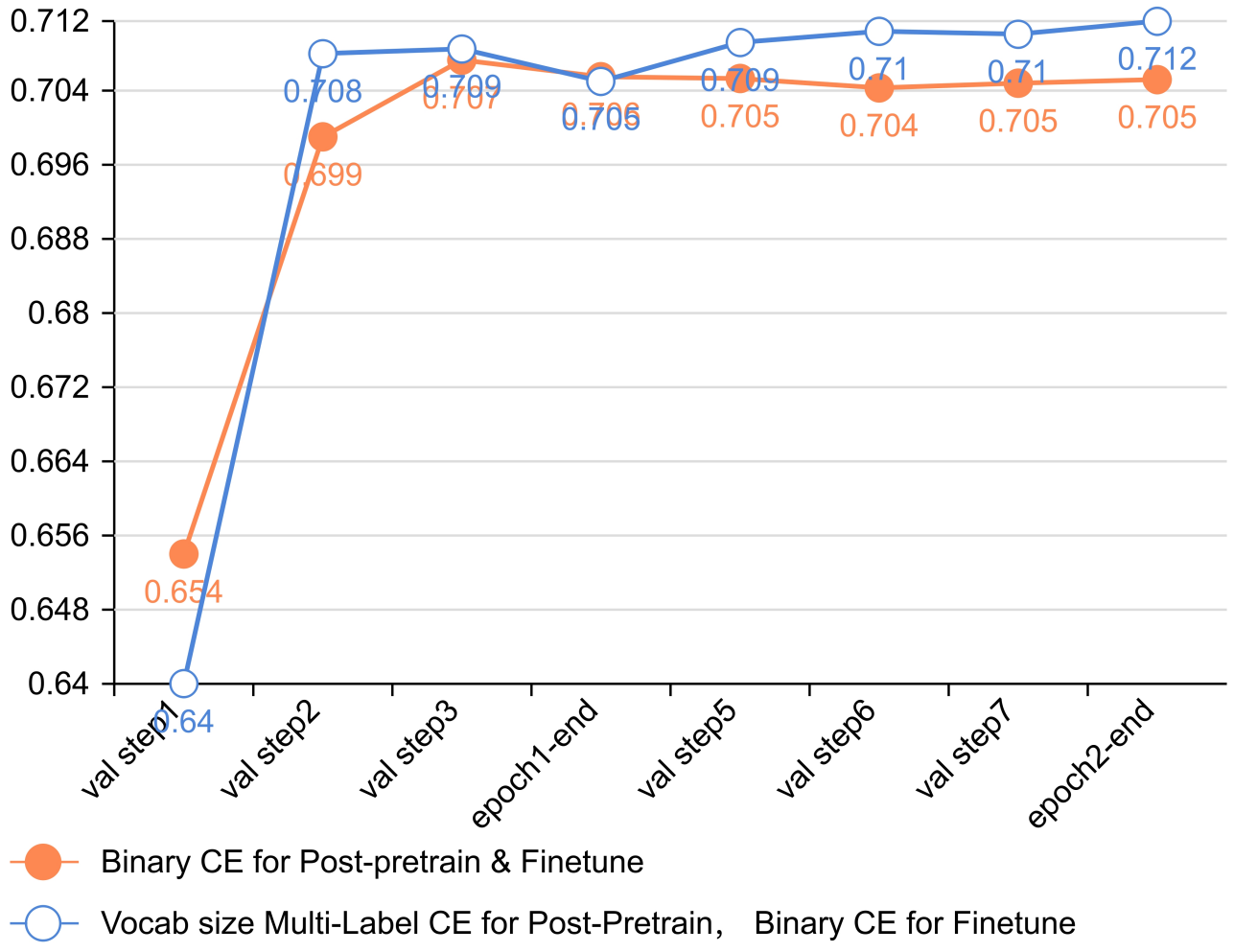
• Apply data augmentation by reversing samples within the same batch

```
():
    () -> :
       ().__init__()
       ......
       .tokenizer = tokenizer
       .a = tokenizer.encode()[]
       .b = tokenizer.encode()[]
    ():
       _, data = data_entry
       text = data[]
       text2 = data[]
       features = .tokenizer(text,padding=,add_special_tokens=,return_length=)
       features2 = .tokenizer(text2,padding=,add_special_tokens=,return_length=)
       labels = .a  data[]==  .b
       labels2 = .a  data[]==  .b
        features[],features[],features[], labels,features2[],features2[],labels2
       ......
```

Why Stage-wise Design of Training Loss

During Post Pretrain, use the entire vocabulary to compute the multi-class cross-entropy for A and B, and during Finetune, use binary cross-entropy. This approach can raise the upper limit of model performance but requires more training steps.

fold	epoch 1 best CV	epoch 2 best CV
0	0.7184	0.7209
1	0.7108	0.7153
2	0.7134	0.7145
3	0.7044	0.7091
4	0.7085	0.7115



Obtaining Specific Token Logits with vllm, Using allowed_token_ids=[a_tok_id,b_tok_id] Alongside logprobs=N

(If only use logprobs=N to select the top N tokens, it's possible that neither Token A nor Token B will be among the top N tokens)

```
a_tok_id = tokenizer(, add_special_tokens=)[][-]
b_tok_id = tokenizer(, add_special_tokens=)[][-]
llm = vllm.LLM(
    cfg.model_dir,
    quantization=,
    tensor_parallel_size=,
......)
sampling_params = vllm.SamplingParams(n=, top_k=, logprobs=, max_tokens=, temperature=,
skip_special_tokens=, allowed_token_ids=[a_tok_id,b_tok_id])
responses = llm.generate(test[], sampling_params, use_tqdm=)
```

Using GPTQ 8-bit as the Final Quantization Solution

float16 CV：0.714
Using GPTQ 8-bit quantization: CV: 0.713, LB: 0.703, PB: 0.714
Using AWQ 4-bit quantization: CV: 0.710, LB: 0.708, PB: 0.709

Some Failed Attempts

- **Rank model**
- **Increase the number of distillation models:** Tried expanding training to Llama-3.3-70B-Instruct , but the effect of distilling four models did not significantly differ from that of three models, with differences not exceeding 0.0005.
- **Filter difficult samples:** Difficult samples mainly come from two situations—model capability insufficiency and inherent biases in the samples that violate universal values. Abnormal samples with obvious errors and biases can cause loss oscillation during training. These biases often originate from specific user biases and differ from the general value system of the sample population. Therefore, I trained six 70B models on the OOF (Out-of-Fold) data from fine-tuning for soft voting, filtering out the top 5%-10% of samples with the largest discrepancy between their composite scores and actual labels, and removed them. The remaining data were used to fine-tune a 14B model. However, there was no significant difference in performance compared to not removing these samples, perhaps indicating room for improvement in the method?

Ideas That Were Not Realized

- **Multi-model fusion > Single model TTA (Test Time Augmentation):** Testing multi-model fusion: A single model can achieve a score of 713, while fusion with gemma might reach 715/716.
- **Dynamic model selection > Single model:** First, use the existing process and different LLMs to train N individual models. Calculate the confidence of each model's answers on OOF (Out-of-Fold) data for each question. Then, train a model selector similar to a Mixture of Experts (MOE) router that assigns each question to the most reliable model for judgment. Load each individual model separately to infer the samples assigned to them (Note: Inference time is difficult to control precisely, and in extreme cases, one model might need to infer all samples).

Inference code：here